

Lifelong Opportunity Guide 88 — Web Developer

Version: 2.0 controlled working master

Language: English

Primary U.S. benchmark: O*NET-SOC 15-1254.00 — Web Developers

Canada comparison: NOC 21234 — Web developers and programmers

Colombia comparison: CUOC 25130 — Desarrolladores Web y multimedia

Review date: 2026-08-22

What this career is

A Web Developer builds, modifies and supports websites and web applications. Depending on the role, the work may include browser interfaces, responsive layouts, client-side logic, server-side services, APIs, databases, authentication, performance, testing, deployment, accessibility and production support.

The title is broad. A front-end developer may focus heavily on the browser experience. A back-end web developer may focus on server logic, APIs and data. A full-stack developer may work across both. Some jobs are primarily content or CMS development; others are application engineering roles with significant cloud, security and database responsibilities.

The United States has a direct current benchmark in **O*NET-SOC 15-1254.00 — Web Developers**, updated for 2026 and marked Bright Outlook. Canada maps directly to **NOC 21234 — Web developers and programmers**. Colombia has a direct comparison in **CUOC 25130 — Desarrolladores Web y multimedia**.

Why web development remains useful

Web systems support commerce, banking, healthcare, education, government, media, internal business systems, customer portals, analytics, identity and software-as-a-service products. The technology stack changes quickly, but enduring skills remain valuable:

- understanding user and business requirements;
- structuring information clearly;
- writing maintainable code;
- communicating with APIs and databases;
- testing behavior across environments;
- protecting data and credentials;
- building accessible interfaces;
- diagnosing performance and production problems;
- using version control and reviews;

- learning new tools without abandoning fundamentals.

Common role families

Front-end developer

Often focuses on:

- HTML;
- CSS;
- JavaScript/TypeScript;
- responsive design;
- browser APIs;
- state and component models;
- accessibility;
- performance;
- React, Angular, Vue or similar frameworks.

Back-end web developer

May focus on:

- server-side code;
- APIs;
- authentication/authorization;
- databases;
- queues/background jobs;
- caching;
- logging;
- deployment and infrastructure interfaces.

Full-stack developer

Works across front end and back end. “Full-stack” does not mean expert in every technology; it usually means capable of delivering across multiple layers with appropriate team support.

CMS/e-commerce developer

May work with WordPress, Drupal, Shopify or other platforms, templates, plugins, integrations, content workflows and performance/security maintenance.

Core web foundations

Frameworks change. Web platform fundamentals remain important.

Learn:

- semantic HTML;
- CSS layout and responsive design;
- JavaScript language fundamentals;
- DOM and events;
- forms and validation;
- URLs;
- HTTP methods/status codes;
- request/response headers;
- cookies and sessions;
- browser storage;
- JSON;
- APIs;
- basic security and privacy concepts;
- accessibility;
- Git/version control.

Semantic HTML

Use HTML elements according to meaning rather than appearance alone.
Good structure can improve:

- maintainability;
- keyboard navigation;
- screen-reader interpretation;
- forms;
- search-engine understanding;
- automated testing.

Prefer native controls where they meet the requirement. A custom interactive component can require significant extra keyboard, focus and accessibility work.

CSS and responsive design

Developers should understand:

- cascade and specificity;
- box model;
- Flexbox;
- Grid;
- sizing and units;
- media/container queries where appropriate;
- responsive images;
- readable typography;

- focus states;
- contrast;
- reduced-motion preferences;
- overflow and reflow.

Do not assume a page is responsive because it looks acceptable on one phone and one desktop screen. Test meaningful viewport ranges and content states.

JavaScript and TypeScript

JavaScript remains central to browser-side web development. Current U.S. O*NET posting data show **JavaScript in 47%** of linked postings and **TypeScript in 22%**.

Important concepts include:

- variables/types;
- functions;
- objects/arrays;
- modules;
- promises and async/await;
- error handling;
- events;
- DOM manipulation;
- network requests;
- state;
- testing and debugging.

TypeScript can add static type checking and improve maintainability, but types do not eliminate runtime validation or security checks.

Frameworks and current technology signals

O*NET employer-posting data for 2025 show strong signals for:

- JavaScript **47%**;
- React **35%**;
- CSS **33%**;
- AWS **27%**;
- HTML **26%**;
- RESTful API **24%**;
- Java **23%**;
- TypeScript **22%**;
- Git **22%**;
- Python **21%**;

- SQL **21%**;
- Node.js **18%**;
- Angular **18%**;
- Docker **16%**;
- Azure **16%**;
- Kubernetes **15%**;
- PostgreSQL **12%**;
- MySQL **10%**;
- PHP **10%**;
- GitHub **10%**;
- Vue.js **9%**;
- GraphQL **8%**;
- Jenkins CI **7%**;
- WordPress and MongoDB **6%**;
- JIRA, JSON and Linux **5%**.

These are market signals, not a checklist that every developer must master.

HTTP and APIs

A web developer should understand:

- GET/POST/PUT/PATCH/DELETE concepts;
- status codes;
- headers;
- content types;
- authentication headers/tokens;
- caching basics;
- pagination;
- rate limits;
- timeouts;
- retries and idempotency where relevant;
- CORS at a conceptual level;
- structured error responses.

Do not expose secrets in browser code. Browser-side code and network requests can be inspected by users.

Authentication versus authorization

These are different questions:

- **Authentication:** who are you?
- **Authorization:** what are you allowed to do?

A logged-in user should not automatically be allowed to access every object or administrative action. Server-side authorization remains essential even if the interface hides controls.

Sessions, cookies and tokens

Developers should follow the application's approved architecture for session handling. Important concerns can include:

- secure cookie attributes;
- token storage;
- session expiration;
- logout/revocation behavior;
- CSRF protections where relevant;
- least-privilege scopes;
- avoiding secrets in URLs/logs.

Do not invent a security architecture without team/security ownership when the application has established standards.

Server-side development

Back-end web development can include:

- request routing;
- validation;
- business rules;
- database access;
- background work;
- caching;
- integrations;
- file handling;
- authentication and authorization;
- logging and monitoring;
- error handling.

Validate untrusted input on the server. Client-side validation improves usability but is not a security boundary.

Databases and SQL

Web applications frequently use relational or non-relational databases. Useful concepts include:

- tables/documents;
- keys and relationships;
- indexes;

- transactions;
- constraints;
- parameterized queries;
- migrations;
- backups/recovery ownership;
- connection management.

SQL injection boundary

Do not concatenate untrusted input directly into database queries. Use parameterized queries/prepared statements or approved ORM/query mechanisms. Input validation is helpful but is not a substitute for safe query construction.

Version control and collaboration

Git skills commonly include:

- clone/pull/fetch;
- branches;
- commits;
- diffs;
- merges/rebases according to team practice;
- pull/merge requests;
- resolving conflicts;
- reviewing changes;
- tags/releases where used.

Never commit passwords, API keys, private certificates or other secrets. If a secret is committed, deleting the line later may not remove it from history; follow the incident/rotation process.

Testing

Useful web testing can include:

- unit tests;
- component tests;
- integration tests;
- API tests;
- end-to-end tests;
- accessibility tests;
- compatibility/browser tests;
- performance tests;
- security tests within authorization.

A passing suite is evidence, not proof that the system is defect-free, secure or legally accessible.

Accessibility

Web developers can improve accessibility through:

- semantic HTML;
- keyboard operation;
- visible focus;
- meaningful form labels;
- useful error identification;
- sufficient contrast;
- zoom/reflow support;
- non-color-only communication;
- alt text and media alternatives;
- screen-reader testing where appropriate.

Automated accessibility tools catch only part of the problem space. Passing a scanner does not establish legal accessibility compliance.

Performance

Performance affects usability, capacity and cost. Relevant areas include:

- payload size;
- images/fonts;
- JavaScript execution;
- rendering;
- network latency;
- caching;
- database queries;
- API latency;
- server capacity;
- CDN use;
- lazy loading where appropriate.

Core Web Vitals can be useful performance/user-experience signals, but no metric or technical change guarantees search ranking, conversion or revenue.

Error handling, logging and monitoring

Applications should fail in controlled ways. Good practice includes:

- useful user-facing errors without exposing internals;

- structured logs;
- correlation/request identifiers where appropriate;
- monitoring and alerts;
- protection of sensitive information in logs;
- clear incident escalation.

Do not log passwords, tokens, full payment data or unnecessary personal information.

Secrets and configuration

Keep environment-specific configuration out of source code where appropriate. Use approved secrets/configuration systems.

Examples of sensitive values:

- database passwords;
- private API keys;
- signing keys;
- OAuth client secrets;
- cloud credentials;
- private certificates.

A `.env` file can be useful locally but should not be assumed secure merely because of its filename. Follow repository ignore rules and organization standards.

Dependencies and package management

Third-party packages speed development but add supply-chain risk. Good practice can include:

- minimize unnecessary dependencies;
- pin/manage versions according to team policy;
- review security advisories;
- update supported packages;
- verify licenses where required;
- avoid abandoned/untrusted packages;
- test updates before production.

Deployment and rollback

A deployment process should define:

- what version is being released;
- environment/configuration;
- tests/gates;

- database migration sequence;
- backup/restore responsibility where relevant;
- monitoring;
- rollback or forward-fix approach;
- who has release authority.

Do not deploy directly to production merely because technical access exists. Follow change/release controls.

Cloud and shared responsibility

AWS, Azure and other cloud platforms are common in web-development postings. Managed services can reduce operational work, but cloud providers do not automatically own every application-security, identity, data, configuration and code responsibility. Follow the provider/service shared-responsibility model and organizational architecture.

Privacy and data minimization

Collect and retain only data needed for approved purposes. Developers should understand the organization's rules for:

- personal information;
- consent/preferences where applicable;
- analytics;
- cookies/tracking;
- retention/deletion;
- exports;
- test data;
- access controls.

Do not invent legal requirements. Escalate privacy/legal questions to the appropriate accountable role.

Secure-development boundary

Security is broader than one checklist. Relevant practices can include:

- server-side validation;
- output encoding;
- parameterized queries;
- authentication and authorization;
- session protections;
- secrets management;
- dependency management;
- secure configuration;

- logging/monitoring;
- security review and testing.

NIST SSDF and OWASP resources are useful references. They do not grant authorization to penetration-test systems. Intrusive scanning, exploitation or destructive testing requires explicit scope and permission.

Responsible AI in web development

AI can assist with low-risk work when policy permits:

- explaining code;
- drafting functions/components;
- refactoring;
- generating tests;
- creating synthetic test data;
- drafting documentation;
- suggesting debugging paths.

Human review remains necessary.

Do not:

- upload proprietary source code, customer data, secrets, credentials or unreleased product information to unapproved tools;
- assume generated APIs/packages exist;
- merge generated code without understanding/testing it;
- accept insecure patterns merely because code compiles;
- ignore dependency or license implications;
- let AI deploy to production outside governance;
- treat generated explanations as execution evidence.

Ethical and professional boundaries

A Web Developer should not:

- deploy without assigned authority;
- hide known production-impacting defects;
- place secrets in source/public repositories;
- bypass authentication/authorization to meet a deadline;
- use production data in personal demos;
- conduct unauthorized security testing;
- claim a site is secure, accessible or defect-free merely because tests passed;
- guarantee SEO ranking, revenue or conversion outcomes;
- publish employer code or private architecture as portfolio material.

Education and entry pathways — United States

O*NET places Web Developers in **Job Zone Three — Medium Preparation Needed**. This often corresponds to vocational/technical preparation, related experience or an associate degree, although employers vary significantly.

Useful learning areas include:

- HTML/CSS/JavaScript;
- one front-end framework;
- server-side programming;
- SQL/databases;
- APIs;
- Git;
- testing;
- accessibility;
- security fundamentals;
- deployment/cloud basics.

CareerOneStop/American Job Centers can help locate training and WIOA-approved programs. Eligibility/funding are not automatic.

Apprenticeship.gov can be used to search current opportunities; the existence of an occupation pathway does not guarantee an opening.

Canada

Canada Job Bank maps Web Developer to **NOC 21234 — Web developers and programmers**.

Current typical requirements include a bachelor's degree in computer science, programming, web development or software engineering **or** a college program in computer science/related field; programming experience is usually required.

Job Bank currently identifies the occupation as **not regulated in Canada**.

Canada wages

National wages updated November 19, 2025:

- **C\$21.48/hour low;**
- **C\$38.46/hour median;**
- **C\$57.16/hour high.**

Canada outlook

The current national 2024–2033 outlook indicates labour demand and supply are expected to be broadly in line. Three-year prospects vary considerably by province/territory; verify the current regional page before making a relocation/training decision.

Colombia

CUOC 25130 — Desarrolladores Web y multimedia directly covers web/multimedia development at competency level 4.

The profile includes analysis, design, programming and modification of web and interactive applications. The guide does not manufacture a representative Colombian national salary from insufficient/nonrepresentative profile indicators.

SENA pathways

Análisis y desarrollo de software

- Tecnólogo;
- **3,984 hours**;
- broad software requirements/design/development/implementation/quality training;
- availability, modality and seats change by cohort.

Desarrollo web con PHP

- complementary virtual training;
- **40 hours**;
- prior programming/HTML knowledge expected;
- focused supplemental web-development practice.

The 40-hour course is not equivalent to the 3,984-hour Tecnólogo.

Latin America and Caribbean

ILO/Cinterfor is a regional vocational-training institution locator. It does not guarantee a current web-development course, scholarship, funding award or seat in every country.

Current wage and outlook research

United States official data

BLS 2025/O*NET Web Developer wages:

Percentile	Annual	Hourly
10th	\$48,100	\$23.12
25th	\$64,230	\$30.88
Median	\$92,650	\$44.54
75th	\$126,230	\$60.69
90th	\$162,290	\$78.03

2024-2034 outlook:

- employment 2024: **86,000**;
- projected 2034: **92,500**;
- growth: **8%**;
- annual openings: **5,400**.

Current non-government U.S. estimate

Indeed's U.S. Web Developer page, updated **August 2, 2026**, reports approximately:

- average **\$86,333/year**;
- low **\$50,037/year**;
- high **\$148,958/year**;
- about **1.4k** observations from job postings over the prior **36 months**;
- displayed cash bonus context **\$2,500/year**.

This is title-specific non-government market context and should not replace official statistics.

Practical learning sequence

Stage 1 — browser foundations

Learn HTML, CSS, JavaScript, accessibility and Git.

Stage 2 — application development

Add a framework, forms, state, APIs, validation and tests.

Stage 3 — server and data

Learn one server-side stack, SQL/database fundamentals, authentication/authorization and secure query patterns.

Stage 4 — delivery

Learn CI/CD basics, environment configuration, logging, monitoring, deployment and rollback.

Stage 5 — specialization

Choose deeper focus in front end, back end, full stack, e-commerce/CMS, accessibility, performance, cloud or secure development.

Safe portfolio projects

Use self-built/open-source/demo systems and synthetic/public data. A strong project can show:

1. requirement/problem statement;
2. responsive semantic UI;
3. API/data layer;
4. validation and error handling;
5. authentication demo where appropriate;
6. tests;
7. accessibility checks;
8. Git history;
9. README and architecture notes;
10. controlled demo deployment;
11. limitations/security assumptions.

Never publish employer source, real customer data, credentials, internal URLs, private architecture or unauthorized vulnerabilities.

Four-week starter plan

Week 1

Build a semantic responsive multi-page site using HTML/CSS; test keyboard use and multiple viewport sizes.

Week 2

Add JavaScript interactions, form validation, fetch a public/demo API and handle loading/error states.

Week 3

Add a small server/API and database or structured local data layer; use parameterized queries and safe configuration.

Week 4

Add tests, README, accessibility/performance notes, version-control history and a controlled demo deployment.

Job-search titles

Search broadly:

- Web Developer;
- Junior Web Developer;
- Front-End Developer;
- Back-End Web Developer;
- Full-Stack Developer;
- Web Application Developer;
- JavaScript Developer;
- React Developer;
- WordPress Developer;
- PHP Developer;
- UI Developer;
- E-commerce Developer.

Questions before accepting a role

- Is the role front end, back end, full stack or CMS/e-commerce?
- Which frameworks/languages are actually used daily?
- Who owns architecture and security decisions?
- How are code reviews and CI/CD handled?
- What is the production-release process?
- How are secrets and environment configuration managed?
- What accessibility standard/process does the team use?
- What testing is expected from developers?
- Who responds to production incidents?
- Is there on-call or after-hours deployment work?
- How is technical debt prioritized?
- What distinguishes junior from senior performance?

Sources and verification links

United States

- O*NET details: [Source link](#)
- O*NET summary: [Source link](#)
- O*NET Job Zone: [Source link](#)
- O*NET wages: [Source link](#)
- O*NET outlook: [Source link](#)
- O*NET technologies: [Source link](#)
- CareerOneStop WIOA: [Source link](#)
- Apprenticeship.gov: <https://www.apprenticeship.gov/>
- Indeed market context: [Source link](#)

Canada

- Job Bank summary: [Source link](#)
- Job Bank requirements: [Source link](#)
- Job Bank wages: [Source link](#)
- Job Bank outlook: [Source link](#)
- Canada training: [Source link](#)

Colombia and Latin America

- OCUPACOL CUOC 25130: [Source link](#)
- SENA Análisis y desarrollo de software: [Source link](#)
- SENA Desarrollo web con PHP: [Source link](#)
- OIT/Cinterfor: <https://www.oitcinterfor.org/statsfp/paises>

Security, AI and accessibility

- NIST SSDF: <https://csrc.nist.gov/pubs/sp/800/218/final>
- OWASP WSTG: [Source link](#)
- NIST AI RMF: [Source link](#)
- Section 508: <https://www.section508.gov/create/>
- WCAG 2.2: <https://www.w3.org/TR/WCAG22/>

Important notice

This guide provides general education and career-planning information. It does not guarantee employment, income, admission, funding, apprenticeship placement, certification, promotion, search ranking, revenue, conversion, security, accessibility compliance or any other outcome.

No independent human certification, professional accreditation, legal review, security assessment, accessibility certification, cloud/vendor certification or certified translation is claimed unless separately documented.

Author and AI assistance

Created and directed by **Alberto “Al” Leiva**. ChatGPT supported research, organization, editing, translation support and document preparation under the author's direction. The author remains responsible for editorial and publication decisions.

License

Unless a file states otherwise, this material is licensed under **CC BY-NC-SA 4.0**.